

SmartTwins : Système multi-agent d'apprentissage du comportement dans un environnement ambiant

RAPPORT DE STAGE MASTER 2
DONNÉES ET CONNAISSANCES

UNIVERSITÉ PAUL SABATIER

Thomas GANDILHON

Encadrant entreprise : Patrick MARQUET

Encadrant laboratoire : Marie-Pierre GLEIZES

Encadrant universitaire : Farah BENAMARA



Table des matières

1	Résumé du stage	3
1.1	Résumé Français	3
1.2	Résumé Anglais	4
2	Remerciements	6
3	Contexte du Stage	7
3.1	Présentation de l'entreprise	7
3.2	Présentation du laboratoire	8
3.3	Présentation du sujet de stage	8
3.4	Objectif du stage	9
3.5	Organisation du travail	10
4	Études du domaine des multi-agents et de travaux sur les intel- ligences ambiantes	11
4.1	Systèmes multi-agents	11
4.1.1	Les agents	12
4.1.2	L'environnement	12
4.1.3	Les systèmes multi-agents adaptatifs	14
4.2	Intelligences ambiantes	15
4.2.1	Apprentissage artificiel adapté aux systèmes complexes par auto-organisation coopérative de systèmes multi-agents	16
4.2.2	Auto-organisation de dispositifs robotiques par la démonstration	17
4.2.3	Approche décentralisée de l'apprentissage constructiviste et modélisation multi-agent du problème d'amorçage de l'apprentissage sensorimoteur en environnement continu : application à l'intelligence ambiante	17
4.2.4	Prise en compte de la dynamique du contexte pour les systèmes ambiants par systèmes multi-agents adaptatifs .	18
5	Conception du Système Smart Twins	19
5.1	Hypothèses	19
5.2	Présentation générale du système Smart Twins	20
5.3	Le module Digital Twins	21

5.4	Préférences utilisateur	22
5.5	Apprentissage qualitatif	22
5.5.1	Qualité	23
5.5.2	Architecture	23
5.5.3	Intentions	24
5.5.4	Interrogation	25
5.6	Apprentissage quantitatif	25
5.7	Articulation des modules	26
5.8	Verrous scientifiques	26
6	Conception et Réalisation du module Préférences Utilisateurs	27
6.1	Première conception	27
6.1.1	Agents Percepts	27
6.1.2	Agents Contextes	28
6.1.3	Agent Head	29
6.1.4	Situations de non coopération	29
6.2	Seconde conception	30
6.2.1	Hypothèses	30
6.2.2	Fonctionnement du système	31
6.2.3	Agents	34
6.2.4	Mécanisme d'apprentissage	36
6.2.5	Adéquation fonctionnelle	38
6.2.6	Coopération entre systèmes	40
6.2.7	Initialisation du système	41
6.3	Sélection du système développé	41
6.4	Outil de visualisation	42
7	Évaluation	43
7.1	Génération des données	43
7.2	Critères d'évaluation	44
7.3	Tests avec un capteur	45
7.4	Tests avec plusieurs capteurs	46
8	Conclusions	48
8.1	Conclusion	48
8.1.1	Bilan	49
8.1.2	Bilan personnel	49
	Appendices	51

Chapitre 1

Résumé du stage

1.1 Résumé Français

Ce stage est une collaboration entre Sogeti France et l'Institut de Recherche en Informatique de Toulouse (IRIT), et plus précisément l'équipe Systèmes Multi-Agents Coopératifs (SMAC). L'objectif de cette collaboration est de permettre à l'entreprise de s'approcher et de s'approprier le paradigme des systèmes multi-agents dans le cadre de leur activité IoT. Le stage a pour but le développement et le test d'un système d'intelligence ambiante basée sur le paradigme des systèmes multi-agents. La conception et le développement de ce système sont faits en collaboration entre l'équipe de recherche et le pôle innovation de Sogeti afin de prendre en compte les contraintes techniques de l'entreprise et les connaissances théoriques du laboratoire.

Différents travaux avec des contraintes et objectifs similaires ont été menés à l'IRIT auparavant, et nous avons débuté la conception du système en se basant sur ces thèses. SmartTwins, proposé dans ce stage, est complexe. Aussi le stage s'est focalisé sur la conception et la réalisation d'une sous-itération fonctionnelle du système.

Tel que nous l'avons conçu le système est composé de différents modules. Le premier module, Digital Twins permet la virtualisation du monde réel dans notre système. Le second module Préférences Utilisateurs est celui chargé d'observer l'environnement pour apprendre les préférences utilisateurs et effectuer des actions pour les respecter. Le module Apprentissage Qualitatif est chargé de détecter les intentions de l'utilisateur par observation de l'environnement et d'apporter cette connaissance au module précédent. Finalement le module Apprentissage Quantitatif est chargé de prédire l'état du système selon son état actuel et l'action qui va être effectuée. Nous avons choisi de développer le module central de ce système, le module Préférences Utilisateurs.

A la fin du développement, le système est fonctionnel sur des cas théoriques avec des données générées et est prêt pour être testé dans des cas concrets. Il représente une base et une brique de conception moteur pour la suite du projet. Sogeti se voit donc maintenant armé pour poursuivre le développement de ce système, et pour le faire converger vers leurs besoins. C’est aussi le début d’une relation entre l’entreprise et le laboratoire qui réunissent des problématiques communes aux deux parties.

D’un point de vue personnel le stage fut très enrichissant. Il m’a permis de me familiariser avec les systèmes multi-agents, en dépassant la théorie et en expérimentant avec leur implémentation. Il m’a aussi conforté dans mon projet professionnel, qui est de réaliser une thèse industrielle, puisque les partenariats entre la recherche et l’industrie sont très stimulants et représentent de véritables challenges.

1.2 Résumé Anglais

This internship is a collaboration between Sogeti France and the Research Institute in Computer Science of Toulouse (IRIT), and more precisely, the co-operatives multi-agent systems team (SMAC). This collaboration’s objective, revolving around this internship, is to allow the company to familiarize and appropriate itself the multi-agent system paradigm in their IoT activity. Thereby, this internship’s objective is to design, develop and test an ambient intelligence system based on the multi-agent system paradigm. The design and the development of this system were conducted by the research team and the innovation team at Sogeti to keep in mind the technical constraints the company is facing, and the theoretical knowledge of the research team.

Several works with similar constraints and objectives have been conducted at the IRIT previously, and we were able to start design the system based on these thesis. The resulting system of this conception was complex, thus a smaller functional iteration of the system was selected as objective for this work.

As we conceived it, the system is divided into several modules. The first module, Digital Twins, allows to virtualize the real world into our system. The second one, User Preferences, is the one in charge of observing and understanding user preferences according to their actions and therefore to act in order to respect them. The Qualitative Learning module is detecting user intentions through environment and user actions, this knowledge is then used by the precedent module. Finally, the Quantitative Learning module predicts the state of the system according to its current state and the next action to be done. The decision was taken to develop the core module of this system, the User Preferences module.

At the end of the development, the system fully operates on theoretical use case with generated data and is ready for real world testing with concret cases. It still represents a good starting point and a conception base for similar works. Sogeti is now prepared to pursue this system development and is now more aware of the constraints and characterists around this system. It is also the beginning of relationship between the company and laboratory, more precisely with the neOCampus project, that gathers shared problematics by both sides.

Finally, on a personnal point of view, this internship was really enriching. It allowed me to familiarize with multi-agent system, by going further than theory and experimenting their implentation. It also strenghened my career project, that is to make an industrial PhD as collaboration between the research world and the industry are highly stimulant and represent real challenges.

Chapitre 2

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué au succès de mon stage et qui m'ont aidé lors de la rédaction de ce rapport.

Tout d'abord j'adresse mes remerciements à mon encadrant Sogeti, Patrick Marquet, qui a rendu ce stage possible, qui m'a permis de le réaliser et pour son aide et son ouverture. Grâce à sa confiance j'ai pu m'accomplir dans les tâches qui m'étaient confiées.

Je tiens également à remercier vivement Marie-Pierre Gleizes, mon encadrant laboratoire et Pierre Glize pour leur aide tout au long de mon travail. Leur conseils et expertises furent un apport précieux et moteur à ce travail. Enfin leur aide à la rédaction de ce rapport s'est révélée indispensable et très bénéfique pour ce document.

Je remercie aussi Maxime Houssin, pour ces longues discussions autour de ce système sur lequel nous travaillons tous les deux. Elles m'ont permis de mieux appréhender mon travail et de mieux comprendre les mécanismes des systèmes multi-agents.

Finalement je remercie mes collègues du pôle innovation Sogeti pour avoir été à l'écoute pour m'avoir apporté leur aide tout au long de ce stage.

Chapitre 3

Contexte du Stage

3.1 Présentation de l'entreprise

Ces cinq dernières années ont vu l'essor de l'Internet des Objets (plus communément appelé par son terme anglais "Internet of Things", IoT) et de l'Intelligence Artificielle. Malgré que ces deux domaines semblent voués à cohabiter dans les systèmes complexes, leur rencontre n'est pas tâche aisée et requiert que de nombreuses barrières soient levées.

Sogeti, entreprise de services informatique appartenant au groupe Capgemini, a décidé de se positionner sur ce marché d'abord à l'aide de son pôle Innovation et plus particulièrement celui de Toulouse. C'est dans ce pôle que j'ai effectué mon stage. Les activités du pôle sont diverses et variées avec un focus sur le thème des objets connectés et plus généralement, de l'IoT.

C'est dans le cadre du projet CBIoTS (Cockpit for Big IoT Systems) que j'ai rejoint l'entreprise. En effet, l'Internet des objets peine à trouver sa place dans le monde et son utilité, même si évidente aux yeux de tous, reste à prouver. De nombreuses applications existent déjà et c'est à travers elles que de tels systèmes peuvent réellement exprimer leur potentiel.

Ma mission était donc de démarrer le développement d'une nouvelle brique pour leur système.

3.2 Présentation du laboratoire

L'intelligence artificielle est un des outils les plus puissants de ces dernières années. Les progrès réalisés dans ce domaine ont permis d'automatiser de très nombreuses tâches jusqu'alors très difficiles pour les programmes. Ainsi, c'est tout un pan de l'informatique qui s'ouvre au monde et qui offre de grandes possibilités aux entreprises. Cependant l'intelligence artificielle ne prend rien pour acquis et évolue très vite. C'est pourquoi ce stage fut réalisé en coopération avec l'équipe SMAC (Systèmes Multi-Agent Coopératifs) de l'IRIT dans le cadre du projet Universitaire neOCampus.

Cette collaboration permet à l'entreprise d'accéder aux équipements déjà en place dans neOCampus (dans notre cas, des salles de cours équipées de capteurs) et à l'équipe SMAC de permettre à leur travail sur l'intelligence ambiante de progresser.

L'équipe SMAC est spécialisée dans les systèmes multi-agents, qui sont un des nombreux paradigmes de l'intelligence artificielle. Leurs travaux ont comme point commun d'utiliser les systèmes multi-agents, mais regroupent de nombreux domaines de l'informatique et du monde industriel en général.

Aussi, le projet neOCampus, dirigé par Marie-Pierre Gleizes, directrice de l'équipe SMAC, a pour essence de regrouper de nombreux laboratoires autour du thème commun qu'est la modernisation du campus universitaire. L'internet des objets est un des sujets phares de cette initiative et ma participation dans ce projet était donc évidente.

3.3 Présentation du sujet de stage

Le campus intelligent ou la ville intelligente, par le biais de ses capteurs fixes et mobiles (capteurs ambiants, objets intelligents et connectés), génère en permanence de grandes masses de données liées à l'activité de ses usagers et de ses processus internes. Ces masses de données représentent une mine d'informations pertinentes pour mieux comprendre l'activité du campus et ainsi le gérer plus intelligemment, dans le but d'assurer l'équilibre entre confort maximal et empreinte écologique minimale.

Au sein du pôle innovation Sogeti et dans le cadre de l'un de ses programmes de R&D orienté gestion de systèmes complexes, l'équipe Innovation travaille sur une plateforme spécifique à l'Internet des Objets de contrôle et de commande de système sociotechniques.

De très grandes infrastructures se mettent en place. Elles sont composées de petits appareils offrant une plus grande capacité de captation et d'action que des ordinateurs : des nano-ordinateurs augmentés de plusieurs capteurs et effecteurs, des smartphones, des drones, etc. La distribution de tels appareils permet non seulement d'avoir cette puissance de captation – grand nombre de petits appareils, diversité des capteurs, etc. – mais aussi de pouvoir agir sur et interagir avec l'environnement, dont l'humain peut faire partie.

La plateforme Cloud IoT (CBIoTS) permet la gestion de ces infrastructures : le contrôle avec la captation de l'information et la commande avec les actions à effectuer, qu'ils soient prévus par un utilisateur ou décidés par le système. CBIoTS se veut une plateforme autonome, accompagnant utilisateurs, administrateurs et décideurs pour la gestion de leurs systèmes.

L'objet du stage est de réaliser un prototype d'intelligence artificielle fondé sur les systèmes multi-agents adaptatifs pour apprendre en temps réel l'activité humaine à partir de ces données. Cet apprentissage doit s'effectuer à partir des données brutes de salles de cours hébergées sur le serveur neOCampus sans connaître a priori la typologie des capteurs, ni l'ordre de grandeur des informations fournies.

3.4 Objectif du stage

Il est attendu de cette intelligence artificielle de proposer une application au système développé par le pôle innovation.

Le but est, dans la durée du stage, de développer un système permettant de mettre en oeuvre le paradigme multi-agent dans une application. Cette application devra être une intelligence ambiante qui sera déployée pour validation expérimentale dans les salles de cours équipées par neOCampus.

L'objectif de Sogeti est de pouvoir s'assurer expérimentalement que les décisions et le choix du paradigme multi-agent sont judicieux afin de résoudre le problème qu'ils se sont fixés. De plus, l'accord avec neOCampus leur permet d'avoir accès à un environnement d'essai pour cette application.

L'objectif de neOCampus est de renforcer ses liens avec les industriels du secteur tout en permettant à l'activité de recherche des laboratoires associés (ici l'IRIT) de poursuivre leurs travaux de recherches et d'enrichir leur expérience des résultats de cette application.

Plus concrètement, l'application doit répondre à quelques critères fixés auparavant :

- Effectuer des traitements et des décisions en temps réel
- Maximiser le confort de l'utilisateur
- Fonctionner sans aucune donnée experte sur l'environnement

3.5 Organisation du travail

La durée du stage a été découpée en sprint de dix jours ouvrés, soit de deux semaines. Ainsi, une estimation prévisionnelle des tâches et de la durée de celle-ci fut dressée (Voir Annexe 1). Au cours de chaque sprint les objectifs et le travail effectué furent étudiés, et le sprint suivant était adapté selon les retards et travaux terminés.

Chapitre 4

Études du domaine des multi-agents et de travaux sur les intelligences ambiantes

4.1 Systèmes multi-agents

Nous avons choisi d'utiliser le paradigme des systèmes multi-agents afin de mettre en place la solution.

Un système multi-agent est un système informatique composé de multiples agents interactifs. Les systèmes multi-agents peuvent résoudre des problèmes difficiles voire impossibles pour un agent individuel ou des systèmes monolithiques.

Un système multi-agent est constitué d'agents et de leur environnement. Les agents possèdent différentes caractéristiques importantes. Tout d'abord ils sont autonomes. Ils sont, au moins partiellement, indépendants, conscients et autonomes. Leur vision est locale. Aucun agent n'a de vue globale sur le système car celui-ci est trop complexe pour qu'un agent puisse exploiter cette connaissance. Finalement, les agents sont décentralisés. Aucun agent n'est désigné comme contrôleur.

Nous commencerons d'abord par aborder les agents, puis nous présenterons le fonctionnement d'un système multi-agent en détaillant ses propriétés principales et plus précisément, comment un système multi-agent interagit avec son environnement. Enfin, nous montrerons quelles sont les propriétés additionnelles d'un système multi-agent auto adaptatif.

4.1.1 Les agents

Définition : "Un agent est une entité réelle ou virtuelle, évoluant dans un environnement, capable de le percevoir et d'agir dessus, qui peut communiquer avec d'autres agents, qui exhibe un comportement autonome, lequel peut être vu comme la conséquence de ses connaissances, de ses interactions avec d'autres agents et des buts qu'il poursuit" [1]

Un agent est une entité autonome physique ou logicielle. Il ne possède qu'une représentation partielle de l'environnement dans lequel le système global est plongé. Cette perception partielle dépend des perceptions de l'agent. Un agent est capable d'interagir avec son environnement grâce à ses compétences tout en suivant ses propres objectifs.

Un agent effectue un cycle de type perception, décision, action. Pendant la perception, l'agent reçoit des informations provenant de son environnement. Elle lui permettent de construire une représentation du monde qui va ensuite lui servir à la décision. Elles sont obtenues à l'aide de ses moyens de perception ou lors de la communication avec une source de données. Pendant la décision, l'agent exploite la représentation du monde précédemment construite afin d'accomplir ses objectifs. Il choisit une ou plusieurs actions afin de converger vers son but. Enfin, pendant l'action, l'agent exécute les actions sélectionnées lors de l'étape précédente. C'est durant cette phase du cycle qu'il agit directement sur l'environnement. Par exemple, une action peut être un déplacement, une interaction avec un effecteur, voire l'envoi d'un message à un autre agent.

4.1.2 L'environnement

L'environnement est une composante indispensable au fonctionnement d'un système multi-agent. C'est au travers de celui-ci que les agents s'expriment et agissent. Différentes caractéristiques sont importantes et les déterminer aide grandement à la conception [7] :

- Accessible ou inaccessible : indique si les agents ont accès à l'environnement dans sa globalité ou non.
- Déterministe ou non déterministe : indique si un changement de l'état de l'environnement est dû à l'état précédent couplé aux actions choisies par les agents.
- Statique ou dynamique : indique si l'environnement peut évoluer pendant qu'un agent est en train de prendre une décision.
- Discret ou continu : indique si le nombre de perceptions et d'actions est limité ou non.

Si on place notre vision à un point de vue macroscopique, l'environnement est constitué de tout ce qui est extérieur au système multi-agent. Cependant, si on considère l'environnement d'un point de vue microscopique, en se concentrant sur un agent, l'environnement se voit maintenant constitué des autres agents également.

Il existe un fort couplage entre l'environnement et le système multi-agent. Chaque modification de l'environnement impacte directement le fonctionnement du système. Aussi, chaque action des agents peut potentiellement modifier l'environnement.

L'autonomie

Un système multi-agent est considéré comme autonome. En effet, comme il est constitué d'agents eux-même autonomes. Chacun prenant une décision sans contrôle extérieur. Ainsi, le système multi-agent dans sa totalité ne dépend d'aucun contrôle extérieur.

La distribution

Chacun des agents possède sa propre représentation du monde et son propre processus de décision poursuivant un objectif. Cette objectif n'est pas l'objectif direct du système mais y contribue. Leur somme de connaissances et de décision permet d'atteindre l'objectif macroscopique.

L'émergence

Un système multi-agent est composé d'un ensemble d'agent en interaction. Cet ensemble d'agent peut être homogène, et chaque agent peut posséder un objectif différent. Ces agents, de part leur comportement, n'ont que conscience de la résolution de leur problème, qui à l'échelle du système peut être qualifié de local, ou microscopique. Cependant leurs objectifs locaux furent déterminés dans l'intention de générer un comportement global émergent. Ainsi, à travers leurs interactions directes ou indirectes, ces comportements peuvent en faire apparaitre un nouveau, cette fois ci au niveau macroscopique, ou global. L'apparition de cette organisation ou de ce comportement est appelée émergence.

Ainsi, un ensemble d'agents peut accomplir plus que la simple somme des compétences locales de chaque agent. On parle d'émergence à partir du moment où la connaissance de la fonction locale d'un agent par un acteur extérieur au système ne permet pas d'inférer la fonction globale du système par ce même acteur.

4.1.3 Les systèmes multi-agents adaptatifs

Le paradigme des systèmes multi-agent permet également de créer des systèmes avec des propriétés supplémentaires. Un de ces systèmes s'appelle système multi-agent adaptatif (Adaptive Multi-Agent System, AMAS). Ils ont la capacité de s'adapter à leur environnement à l'aide de comportements coopératifs afin d'atteindre l'objectif. Cette coopération permet l'auto-organisation.

Coopération

La coopération est le comportement le plus important de tels systèmes. Ce sont les règles qui régissent cette coopération qui va influencer sur l'émergence et le type d'adaptation. Parfois les agents choisiront de rompre cette coopération voire de la consolider. Cette dynamique interne au système permet l'émergence d'organisation complexe et de comportements singuliers. La coopération est la raison de l'adaptabilité de ces systèmes.

Adéquation fonctionnelle

L'adéquation fonctionnelle est un concept qui assure qu'un système puisse répondre à l'objectif qui lui est fixé. Sa définition formelle stipule : "Un système fonctionnellement adéquat est un système qui n'a aucune activité antinomique avec son environnement". Dans la pratique, un système est considéré adéquat fonctionnellement si un observateur externe au système, reconnaît que le système effectue la tâche ou atteint l'objectif qui lui incombe. Il est désirable que le système puisse lui-même déterminer son adéquation. Néanmoins, comme présenté précédemment, chaque agent n'ayant qu'une perception locale, il est difficile de leur faire évaluer cette adéquation.

[2] Il a été démontré qu'un système dont les agents sont tous dans un état coopératif est fonctionnellement adéquat.

Théorème de l'adéquation Fonctionnelle : Pour tout système fonctionnellement adéquat, il existe au moins un système à milieu intérieur coopératif qui réalise une fonction équivalente dans le même environnement.

L'expression système à milieu coopératif signifie que toutes les interactions entre les parties qui composent le système sont coopératives.

Autrement dit, on peut déduire de ce théorème qu'à toute solution calculable à un problème, on peut y attribuer un système multi-agent coopératif qui la réalise. Ce système se doit de n'être constitué que d'agents coopératifs.

Situations de non coopération

Souvent, dans le fonctionnement des agents certains blocages peuvent apparaître au moment de leur coopération. Ces blocages sont appelés des situations de non coopération. Elles peuvent provenir lors d'erreurs dans la perception, dans la décision ou dans l'action. Ces situations ont été classifiées en plusieurs types afin de faciliter leur identification :

- Incompréhension : l'agent ne peut pas extraire l'information du signal perçu.
- Ambiguïté : l'agent peut interpréter le signal perçu de plusieurs manières.
- Incompétence : l'agent ne peut parvenir à aucune décision à partir de ses connaissances actuelles.
- Improductivité : les décisions de l'agent n'aboutissent à aucune action.
- concurrence : les décisions de l'agent n'aboutissent à aucune action.
- Conflit : l'action est incompatible avec celle d'un autre agent.
- Inutilité : l'action n'a aucune conséquence sur son environnement.

Afin de résoudre ces situations, les agents doivent être capables de les détecter afin que la résolution soit locale plutôt que de faire intervenir un contrôle global. Cette résolution demande souvent que l'organisation du système soit modifiée. Les agents disposent de trois moyens afin de revenir à un état coopératif :

- Ajustement : l'agent modifie son comportement interne en ajustant ses paramètres internes
- Réorganisation : l'agent change ses relations avec les autres agents
- Ouverture : l'agent décide de s'auto-détruire ou bien de créer un nouvel agent.

Le choix de la résolution des situations non coopératives est fait lors de la conception.

4.2 Intelligences ambiantes

De nombreux travaux sur les intelligences ambiantes ont été menés. Nous allons ici nous intéresser à quatre thèses.

4.2.1 Apprentissage artificiel adapté aux systèmes complexes par auto-organisation coopérative de systèmes multi-agents

La première thèse [6] présente la conception d'un système dont le but est d'apprendre le comportement d'un utilisateur dans un environnement à l'aide des actions qu'il effectue. A la suite de l'apprentissage du comportement de l'utilisateur le système va pouvoir utiliser cette connaissance pour reproduire les actions.

Les circonstances et l'environnement de ce système l'ont poussé à décliner le système en dispositifs. Nous ne nous intéresserons qu'au fonctionnement des dispositifs ici. Celui-ci fonctionne à l'aide de cinq agents différents :

- Agent capteur : cet agent est en charge de la propagation des données dans le système, il fait le lien entre le capteur physique et le système.
- Agent effecteur : cet agent est en charge de la propagation des données concernant l'état de l'effecteur auquel il est lié ainsi que de modifier l'état de l'effecteur.
- Agent donnée : cet agent est en charge de la propagation d'une donnée vers les agents contextes. Il devra garder trace de l'utilité de cette donnée pour les agents contextes auxquels il est lié.
- Agent contrôleur : cet agent décide quelle action sera réalisée sur son agent effecteur
- Agent contexte : cet agent propose une action particulière à réaliser selon les données qu'il reçoit.

Ce système introduit deux fonctionnements que nous avons identifiés comme fondamentaux dans ce genre de systèmes.

Tout d'abord, la séparation entre les acteurs physiques et les acteurs virtuels. Ceci est effectué par les agents capteurs. Ainsi, chaque agent capteur peut être considéré comme le jumeau virtuel du capteur physique. Cela permet une indépendance entre le système et son penchant réel, physique, concret.

Ensuite, la notion d'agent concept est extrêmement importante. Le fait qu'un agent soit témoin d'une situation et qu'il soit ensuite le garant de l'action qu'elle a nécessité est important. C'est la manière dont le système va pouvoir apprendre le comportement de l'utilisateur.

Ces deux idées sont deux parties importantes du système que nous avons choisi de développer.

4.2.2 Auto-organisation de dispositifs robotiques par la démonstration

Ce travail [8] développe un système très similaire au précédent. Ici seulement trois agents sont utilisés :

- Agent Perception : cet agent construit et acquiert des données et connaissances qu'il va ensuite partager aux agents contextes
- Agent Contexte : le rôle de cet agent est de proposer une action à réaliser lorsque le contexte semble favorable.
- Agent Input : le rôle de cet agent est de réaliser une action proposée par un agent contexte lorsque nécessaire.

Le but du système proposé est d'apprendre un comportement par démonstration. Ainsi, le système est spectateur des activités d'un utilisateur et apprend à reproduire ces actions dans des contextes similaires.

Les agents percepts sont chargés de construire une représentation de l'environnement. Pour ce faire ils utilisent un mécanisme d'AVT (Average Value Tracker). Le principe de ce mécanisme est simple : il utilise une fenêtre qui possède une valeur maximale et une valeur minimale. Il utilise également l'ancienne valeur de l'observation et la valeur courante. Ainsi à chaque moment, l'agent percept possède la valeur de son observation, et les bornes connues de cette observation.

Les agents contextes proposent une action lorsqu'ils sont considérés comme valides. Un agent contexte est considéré comme valide lorsque l'intégralité de ses AVT sont valides. C'est à dire lorsque toutes les valeurs des observations, sont comprises entre les bornes. Les valeurs des observations lui sont transmises par les agents percept. Une fois un agent valide, il transmet son action à l'agent input qui lui est attribué.

Les agents inputs réalisent les actions que les agents contextes proposent.

Ensuite, à l'aide de la démonstration des utilisateurs, des agents contexte sont créés afin de correspondre au comportement souhaité. Les agents contextes sont supposés fusionner lorsque leurs plages de validités sont les mêmes et que leur actions concordent.

4.2.3 Approche décentralisée de l'apprentissage constructiviste et modélisation multi-agent du problème d'amorçage de l'apprentissage sensorimoteur en environnement continu : application à l'intelligence ambiante

Ce travail [5], propose un mécanisme de prédiction et de décision basé sur un système multi-agent. Le but est ici d'apprendre et de détecter l'occurrence de d'évènements. Lorsque les évènements sont reliés ils constituent des motifs.

Le système est constitué de trois agents différents :

- Agent découper : cet agent est chargé de découper les données continues en entrée en événements. Ils produisent donc un ensemble d'événements ensuite envoyés.
- Agent associer : cet agent est chargé d'associer les événements produits par les agents découper. Une fois les motifs associés entre eux, ils forment un motif.
- Agent similarité : cet agent compare les motifs qui lui sont envoyés par les agents afin de savoir s'ils correspondent avec des motifs déjà rencontrés.

Le but du système est de généraliser des scénarios donnés par l'utilisateur, de savoir les reconnaître afin d'appliquer une action déterminée à l'avance.

4.2.4 Prise en compte de la dynamique du contexte pour les systèmes ambiants par systèmes multi-agents adaptatifs

Cette thèse [4] repose sur le même principe que la seconde [8]. Cependant, ici l'idée d'aligner plusieurs de ces systèmes afin de gérer la décision de chacune des actions. Chaque instance de système doit décider s'il faut ou non effectuer une action particulière. Ils prennent en entrée les sorties des autres systèmes afin de pouvoir capter des comportements complexes.

Chapitre 5

Conception du Système Smart Twins

La première partie du travail a consisté en la délimitation des objectifs et de la à la définition du système Smart Twins. Le but ici n'était pas de concevoir le système qui allait être développé pendant le stage, mais de définir l'architecture pour répondre aux objectifs d'apprentissage en temps réel de l'activité humaine.

5.1 Hypothèses

Un certain nombre d'hypothèses ont été formulées à l'origine du travail afin de mieux délimiter les tâches du système.

Tout d'abord, même si le système doit être résistant au bruit, il ne le sera pas face aux données erronées. Par exemple, la panne d'un capteur pourrait grandement impacter la qualité du service. Aussi, il est important de s'assurer que le système possèdera suffisamment d'informations pour effectuer la tâche pour laquelle il est déployé. Nous considérerons donc que le système possèdera toujours suffisamment de perceptions pour résoudre sa tâche.

Le but étant de comprendre les préférences d'utilisateurs dans son environnement, certaines hypothèses sur les utilisateurs ont été faites également. L'utilisateur doit avoir un système de raisonnement rationnel et ses actions doivent être la manifestation d'un modèle interne. Il faut que le système possède au moins autant d'informations que l'utilisateur pour pouvoir correctement effectuer sa tâche.

ARCHITECTURE

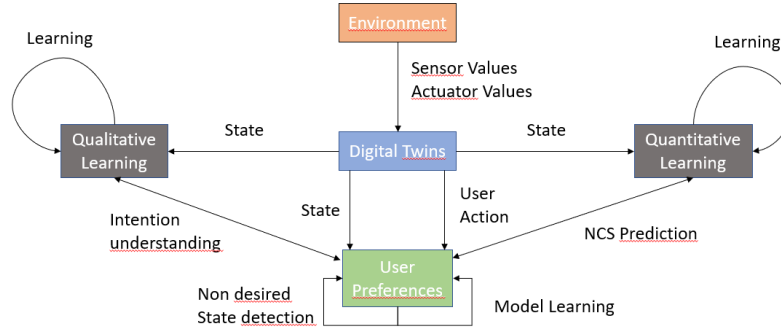


FIGURE 5.1 – Système Smart Twins

5.2 Présentation générale du système Smart Twins

Le système est divisé en 4 modules différents, chacun remplissant un rôle particulier. Les modules ne sont pas indépendants, certains nécessitent les résultats des autres modules. Cependant, ils ont été définis de sorte à pouvoir être développés de manière itérative.

- **Digital Twins** : ce module fonctionne comme une virtualisation du monde physique. Il représente les capteurs réels dans le système et est chargé de diffuser leurs valeurs.
- **Préférences utilisateurs** : ce module est chargé d'apprendre les préférences utilisateurs à l'aide des observations qui lui sont communiquées par le module Digital Twins.
- **Apprentissage qualitatif** : ce module est chargé de découvrir l'impact qualitatif des actions sur l'environnement, toujours à l'aide des observations données par le module Digital Twins.
- **Apprentissage quantitatif** : ce module est chargé d'apprendre et de prédire les impacts, quantifiés, des actions sur l'environnement.

Par la suite, nous allons décrire ces quatre modules et en présenter les fonctionnalités.

5.3 Le module Digital Twins

Comme présenté, l’environnement de notre système possède une dimension réelle. En effet, les décisions et actions de notre système sont ensuite traduites par de vraies modifications de cet environnement. Il est cependant inconcevable que les éléments réels (capteurs et actuateurs) soient inclus directement dans le système, pour des raisons de surcharge réseau et de puissance de calcul disponible sur les capteurs.

Une sur-couche est donc appliquée. Cette sur-couche est un système qui contient un certain nombre d’agents nommés, Digital Twins. Comme leur nom le laisse entendre, ces agents ne sont que des copies, jumeaux, de leur réalité physique. Leur rôle est d’assurer la liaison entre le système et son environnement.

Leurs fonctions et objectifs sont multiples. La première est de mettre à disposition l’état de leur jumeau réel. Dans le cas d’un capteur, cela représente sa valeur actuelle. Pour les actuateurs, c’est également leur état, mais aussi, l’information sur le fait que cet état a été engendré par un comportement humain, ou logiciel. Ce comportement est indispensable car il permet de différencier le feedback utilisateur, de la simple conséquence de nos actions. L’objectif de ce module est de permettre au reste du système de pouvoir posséder, des données à jour à tout moment.

Ce module est séparé du reste du système, puisqu’il a une forte contrainte technique. Il doit communiquer avec les objets réels et en fonction des implémentations, les technologies (Framework et APIs) utilisées peuvent varier. Cependant, le rôle de ce système reste le même.

Il existe plusieurs stratégies de diffusion des données dans un système :

- diffuser les données à intervalles réguliers, cependant, cela implique que la fréquence du besoin est stable et définie.
- donner la possibilité aux autres modules nécessitant ces données, la possibilité de récupérer les données lorsqu’ils en ont besoin. Ainsi chaque module effectuera des requêtes et se voit retourné les données qu’il a demandé.
- permettre au module de lui-même notifier les autres modules lorsque les données sont modifiées.

Comme le présente la figure 5.2 le module Digital Twins peut être représenté comme un vecteur, et pour chacun de ses indices, il contient la valeur du capteur (Sensor), actuateur ou capteur global (Global Sensor) associé.

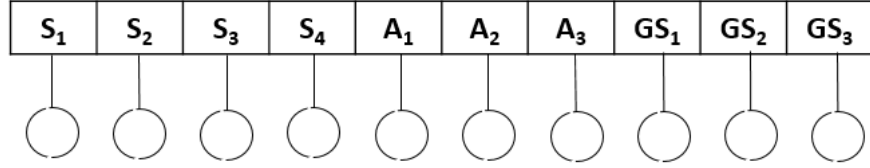


FIGURE 5.2 – Digital Twins

5.4 Préférences utilisateur

Ce module est responsable de l'apprentissage des préférences utilisateurs. Il est le garant de la qualité du service du système. Son but est de comprendre et d'apprendre les préférences des utilisateurs par observation de leur actions dans l'environnement. C'est à travers cette observation qu'il va pouvoir essayer de lier les contextes des actions à celle-ci.

Il lui faudra également prendre des décisions. En effet c'est ce module qui, à l'aide des autres module, va décider de l'action à prendre afin de respecter les préférences apprises et les faire valoir. Son raisonnement sera fondé sur ces observation mais aussi sur les interactions qu'il aura avec les autres modules.

Les différentes conceptions possibles de ce module et leur implémentation seront discutées dans le chapitre 5.

5.5 Apprentissage qualitatif

Le troisième module de ce système est le module d'apprentissage qualitatif. L'objectif de l'apprentissage qualitatif est de modéliser ce que la théorie de l'apprentissage constructiviste appelle : « l'apprentissage sensorimoteur ». Le but de ce module est de comprendre l'environnement dans lequel le système évolue. Il va ainsi apprendre des actions faites dans l'environnement, observer leurs conséquences, et ainsi être capable de devenir (de manière qualitative) quelle action aura un impact sur chacun des valeurs.

5.5.1 Qualité

Quand nous définissons l'apprentissage comme qualitatif, nous faisons référence à la qualité des variations qu'une variable peut subir. Une vision simple, mais suffisante, dans le cadre de l'intelligence ambiante, est qu'une variable peut augmenter, diminuer, ou rester stable. Nous avons décidé de ne pas représenter la stabilité et de se placer dans l'hypothèse du monde fermé, afin que l'absence d'informations, représente l'absence d'influence, et donc de stabilité.

Dans la suite, toute référence à la qualité d'une variation, ou à un apprentissage qualitatif, sera lié à ce concept.

Le choix de limiter l'apprentissage à son aspect qualitatif, à l'inverse de l'aspect quantitatif, est que la quantité de ces variations n'est pas, encore, nécessaire à la compréhension des relations présentes entre les métriques de notre environnement, et qu'une telle restriction, rend le problème plus abordable.

5.5.2 Architecture

Aucune architecture précise n'a été décidée pour l'instant. Le problème de détection d'anomalies dans des données continues, ainsi que la détection de corrélations et causalités, sont des domaines de recherche à part entière. Ces problèmes en soulèvent bien d'autres plus complexes, et aucun système simple ne permet de résoudre cette tâche rapidement. Cependant, il nous semble que l'approche AMAS peut permettre de fournir une solution qui, bien que basée sur des techniques et méthodes mathématiques, permet de générer de la connaissance intelligible.

Le système n'est qu'observateur de l'environnement. Il apprend des interactions au sein même de celui-ci, et la source des actions n'a pas d'importance. Une même action effectuée par un utilisateur ou un logiciel, dans des contextes similaires, aura toujours le même impact, d'un point de vue qualitatif, sur l'environnement. Il prend donc en entrée l'intégralité des valeurs fournies par les digital twins.

Dans notre ébauche de conception, le système est composé de deux types d'agents :

- Agents influences : Ces agents sont chargés de représenter l'influence d'un actuateur sur un capteur. Il ne peut en exister qu'un seul par couple. Son rôle est de témoigner de l'impact de la variation de la valeur de l'actuateur sur la variable du capteur. Cet impact reste qualitatif, les valeurs n'ont que peu d'intérêt dans ce cas.
- Agents patterns : Ces agents sont chargés de représenter l'effet d'une variation donnée sur un actuateur sur la valeur d'un capteur. Ces deux variations sont qualitatives. Plusieurs agents patterns peuvent se contredire. Ils n'ont pas conscience du contexte, et des contractions entre agents patterns sont normales et tout à fait acceptables.

Étant dans le cadre des AMAS, ce système sera amené à évoluer et "épouser" les variations de notre environnement. Des agents influences peuvent se créer si des corrélations sont détectées, et certain peuvent se détruire si jamais elles s'avèrent n'être plus véridiques. Ainsi, cette architecture permettra, théoriquement, au système d'apprendre qualitativement l'environnement.

5.5.3 Intentions

Chacun de ces causalités, représente en fait une manière pour l'utilisateur de modifier son environnement. A travers chacune de ses actions, l'utilisateur exprime une intention. L'intention d'un utilisateur est le désir de modifier l'environnement vers un état qui satisfasse son modèle intérieur. Une intention, est la volonté de modifier une ou plusieurs variables.

La détection de causalités entre actionneurs et capteurs, permet de détecter ces intentions chez l'utilisateur. Pour chaque action observée, nous sommes capables de détecter toutes les variables que cette action peut modifier. Nous pouvons alors supposer que l'utilisateur tente de modifier une de ces variables. Chacune de ces modifications qualitatives, représente une intention. Par exemple, l'ouverture de la fenêtre par l'utilisateur peut être traduit par l'intention de l'utilisateur de modifier la température de son environnement.

Le premier problème rencontré lors de cette traduction, est que bien souvent, chaque actionneur peut modifier plusieurs variables. Il semble raisonnable de penser que l'utilisateur n'a pas conscience de tous ces changements (du moins dans son intention) voire, décide de déplacer certaines valeurs hors de son champ de confort, pour satisfaire d'autres contraintes. Il faut donc, avec le temps, savoir rationaliser les véritables intentions de l'utilisateur.

Le second problème lié à cette traduction, est qu'elle est souvent multiple. Pour reprendre l'exemple de la fenêtre, celle-ci est capable de modifier la température de plusieurs manières différentes. Elle peut soit l'augmenter, soit la réduire. Savoir dans lequel des deux cas nous nous trouvons, implique de s'intéresser aux valeurs de la température intérieure, mais pas seulement. Il faut également prendre en compte la température extérieure. Cela sort alors du cadre de l'apprentissage qualitatif, et nécessite de nouvelles informations comme nous le verrons plus tard.

5.5.4 Interrogation

Le système étant clairement compréhensible par un observateur extérieur, il est possible de lire son état ou même de l'interroger. Il devient donc possible d'effectuer des traductions action <intention, intention >action. Cette intelligibilité de la connaissance et son interrogation, sont deux caractéristiques très puissantes qui nous permettent de pouvoir utiliser ce système depuis le second module présenté, celui chargé de l'apprentissage des préférences utilisateurs.

5.6 Apprentissage quantitatif

Ce quatrième et dernier module, représente la partie manquant de l'apprentissage. En effet, il est nécessaire d'avoir une idée de la quantité d'une modification des valeurs de l'environnement, afin de déterminer quelle est la réelle intention de l'utilisateur.

Ce module pourrait résoudre un des problèmes de l'apprentissage qualitatif qui est son incapacité à prendre en compte le contexte, et fournir, pour une action donnée et un contexte, les intentions des utilisateurs, sans contradictions.

Nous avons dans l'idée ici un système extrêmement efficace, les réseaux de neurones. Ce type d'apprentissage a la capacité de détecter les corrélations et les causalités dans des jeux de données avec un efficace étonnante. Ils combinent apprentissage qualitatif et quantitatif. Cependant, leur plus grande faiblesse est leur opacité.

La connaissance engrangée par les réseaux de neurones n'est pas intelligible et n'est pas compréhensible par un observateur externe. C'est pourtant dans les poids de ce réseau que s'exprime l'apprentissage qualitatif. Ils sont néanmoins très efficaces dans le cadre d'apprentissage quantitatif, notamment pour prédire l'état d'un système étant donné son état actuel.

Malgré tout, les réseaux de neurones s'adaptent très mal à notre environnement qui se veut dynamique et dont le nombre de valeurs peut varier.

Considérons tout de même qu'une solution nous permette d'obtenir cette connaissance. Nous pourrions ainsi interroger ce module afin de lui demander de prédire l'impact d'une action sur un système étant donné un état courant. Il serait ainsi possible de savoir si dans un cas précis, l'ouverture de la fenêtre, va augmenter ou réduire la température. De plus, il serait même possible d'anticiper la valeur future de cette variable (convergence de la température intérieure vers la température extérieure).

La possession d'une telle connaissance, clos donc la boucle de notre système. Nous sommes maintenant capables, pour une liste d'actions données, déterminer lesquelles amèneront à un état désirable.

5.7 Articulation des modules

Le but de ces modules étant défini, nous allons maintenant étudier la manière dont ils s'articulent et comment, ils communiquent pour atteindre l'objectif du système.

Tout d'abord, le système reçoit en entrée les valeurs réelles envoyées par les éléments physiques de l'environnement. Ces valeurs sont relayées dans le système par les digital twins.

Chaque module utilise ces données en entrée. Dans le cas d'une action utilisateur, le module Préférences, consulte ses agents contextes, et dans certains cas, en créer un pour correspondre à cette action. Cependant, au lieu d'exprimer une action, l'agent contexte traduit celle-ci en une liste d'intentions liées à cette action. Avec le temps cette liste évoluera.

A l'aide de cette liste d'intentions, il est possible de « remonter » dans le système d'apprentissage qualitatif pour trouver toutes les actions satisfaisant ces intentions ou au moins une d'entre elles.

Finalement, cette liste est évaluée par l'agent Head à l'aide de la connaissance de l'apprentissage quantitatif afin de choisir l'action ou l'ensemble d'action qui répondent au mieux au profil de l'utilisateur, en prenant garde à ne pas mener le système dans un état plus critique.

5.8 Verrous scientifiques

Résoudre un problème tel que l'apprentissage de préférences utilisateurs sur des variables inconnues implique de résoudre un certain nombre de sous-problèmes.

Tout d'abord il est nécessaire de pouvoir s'adapter à l'environnement, et celui-ci est en perpétuel changement. Ainsi il faut que le système puisse représenter les préférences de l'utilisateur selon l'environnement, sans pour autant perdre la possibilité de modifier les connaissances sur l'environnement, comme par exemple le profil a priori des variables d'entrées.

Ensuite, afin de pouvoir respecter la caractéristique citées précédemment, il faut que la représentation des préférences permette que le système puisse toujours fonctionner.

Comment notre objectif sont les intelligences ambiantes, il est nécessaire que le système fonctionne en temps réel.

Finalement, comme le système doit évoluer dans un contexte socio-technique il serait intéressant qu'il soit capable de détecter les préférences utilisateurs pertinentes rapidement, c'est-à-dire, en peu d'exemples.

Chapitre 6

Conception et Réalisation du module Préférences Utilisateurs

Au cours de nos recherches, deux conceptions différentes ont émergé. Par la suite, nous allons décrire ces deux conceptions et justifier le choix de l'une des deux.

6.1 Première conception

Cette première conception est très inspirée des travaux fait par d'anciens doctorants de l'IRIT [8] [6] [5] [4]. L'intuition de cette approche est de retenir les contextes dans lesquels les actions sont faites afin de pouvoir les reproduire dans des contextes similaires.

Ce module est en fait un AMAS et est composé de 3 types d'agents.

6.1.1 Agents Percepts

Ces agents sont chargés de recevoir en entrée les valeurs des Digital Twins décrits précédemment. En plus de percevoir les données, ils doivent également les "cartographier" simplement. La cartographie de leurs valeurs est en fait un simple mécanisme de mémoire des valeurs maximales et minimales rencontrées. Ce mécanisme est utile pour le second type d'agents.

Il peut être envisageable de déplacer ce comportement des agents percepts vers les Digital Twins. Cependant, comme on souhaite conserver une indépendance technologie / programme. Circonscrire ce comportement dans cette partie du système semble plus judicieux.

6.1.2 Agents Contextes

Les agents contextes sont les représentant des préférences utilisateurs. Lorsqu'une action est effectuée par un utilisateur, il est nécessaire pour notre système de la considérer comme pertinente. Ainsi, si la même situation venait à se représenter dans le futur, il semble pertinent de reproduire la même action. L'agent contexte lie donc un contexte à une action. N'étant pas dans la capacité d'effectuer l'action lui-même, il va transmettre sa proposition à l'agent head.

Chaque agent contexte représente donc une situation de référence dans laquelle un utilisateur a effectué une action donnée. On peut donc traduire la proposition d'un agent contexte à l'agent head par "la situation actuelle est semblable à cette situation où l'utilisateur a effectué cette action. Si cette action est également effectuée maintenant, le système devrait remplir son rôle". Une situation n'est en fait qu'une combinaison de variables perçues par le système.

Cependant, retenir une seule valeur pour chacune de ces variables ne permet pas à l'agent contexte de bien représenter les préférences de l'utilisateur. Au-delà de la non-applicabilité au monde réel (la température varie énormément en petite quantité chaque seconde), il serait compliqué de reconnaître les contextes similaires. Il est donc préférable de travailler avec des intervalles. Chaque agent possède donc autant d'intervalles que de perceptions (figure 6.1). Chacun de ses intervalles représente les valeurs sur lesquelles cet agent est valide.

Il produira une proposition lorsque l'intégralité de ses intervalles seront valides. Un intervalle est considéré comme valide, lorsque la valeur réelle à laquelle il est lié se trouve entre ses deux bornes.

Un agent contexte a besoin de communiquer afin de mener à bien son objectif. Il est donc en communication avec les agents percepts qui lui fournissent les données dont il a besoin pour détecter lorsque son contexte apparaît.



FIGURE 6.1 – Agent Contexte

6.1.3 Agent Head

L'agent head est responsable de la décision finale. C'est lui qui compare les propositions faites par les agents contextes, et qui décide laquelle réaliser. Il est en contact avec tous les agents contextes et ceux-ci lui proposent selon la situation courante, une liste d'actions possibles.

Il est capable de dire à chacun des agents contextes qui l'ont contacté lorsque leur action a été sélectionnée, et inversement lorsqu'elle est choisie. Il est également capable de communiquer avec les digital twins, afin de leur faire parvenir l'action que l'on souhaite effectuer sur le système.

Finalement, il possède la compétence de déterminer quelle est la meilleure action à réaliser, parmi un ensemble d'actions.

6.1.4 Situations de non coopération

Les situations de non coopération (SNC) sont des situations dans lesquelles le système a failli à suivre les préférences des utilisateurs et doit donc apprendre de cette erreur. Nous allons ici citer les SNC les plus probables et qui apparaîtront durant le fonctionnement du système

Réalisation d'une action par l'utilisateur

Si un utilisateur effectue une action alors que le système n'en a pas fait, c'est qu'elle ne faisait pas partie des propositions d'actions envoyées par les agents contextes. Le système est donc dans l'incapacité de réaliser une action à la place de l'utilisateur dans ce contexte.

Elle peut être résolue par la création d'un agent proposant l'action faite par l'utilisateur dans le contexte courant.

Aucun agent contexte valide

Si aucune agent contexte ne propose d'action, le système est face à un problème d'improductivité. Pour résoudre cette situation, il peut être nécessaire d'étendre les bornes d'un agent contexte proche de la situation actuelle afin qu'il soit valide dans la situation courante. Ou de créer un agent supplémentaire qui stipule de faire l'action vide dans ce contexte (ou de maintenir l'état des actuateurs).

Deux agents contextes proposent deux actions différentes

Si deux agents contextes proposent deux actions différentes il faut alors choisir laquelle effectuer. Cela peut être la conséquence de création successives d'agents.

Pour faire ce choix entre plusieurs actions, il faut une valeur associée à chaque agent qui caractérise la confiance de chaque agent. Cette valeur évolue avec le temps (lorsque l'agent contexte voit son action choisie ou non).

Il est également envisageable d'utiliser un système d'auto-organisation. Dans un tel système, chaque agent conserve une liste des agents qu'il juge supérieurs et ceux inférieurs. Sa décision sera prioritaire par rapport à celle des agents inférieurs.

Deux agents contextes proposent la même action

Si deux agents contextes proposent la même action, il est nécessaire de modifier leur niveau de confiance. Ainsi seul l'agent avec le niveau de confiance le plus élevé sera conservé. Sa confiance augmentera et celle de celui non sélectionné diminuera.

L'action proposée est contredite

Si une action proposée par un agent contexte est acceptée, mais ensuite contredite par une action de l'utilisateur (action contraire), il est nécessaire d'adapter la fenêtre de l'agent contexte afin de ne plus proposer cette action dans ce contexte. Il faut donc soit faire varier sa fenêtre d'activation soit notifier à l'agent qu'un autre agent avait raison, et ainsi le placer dans sa liste d'agents supérieurs.

6.2 Seconde conception

Cette seconde conception est basée sur [3]. Nous sommes partis du système existant dans l'équipe SMAC et nous avons ici réutilisé le système développé par cet article et nous l'avons adapté à notre problème.

6.2.1 Hypothèses

Comme présenté par [3] le système tente de séparer l'ensemble des contextes rencontrés en deux ensembles distincts. Cette séparation se fait à l'aide d'une fonction linéaire. Ainsi la première hypothèse que le système fait, c'est que cette séparation linéaire existe.

Aussi même si le système possède une certaine résistance au bruit, il est tout de même préférable que les données en entrée soient assez fiables car tous les contextes rencontrés seront interprétés de la même manière, avec ou sans bruit. Le niveau de bruit n'est donc pas détecté mais un faible niveau ne sera pas impactant sur le système.

Finalement, ce séparateur linéaire fonctionne par apprentissage supervisé. Il n'est donc pas envisageable de l'utiliser si aucun feedback n'est disponible. Dans notre cas le feedback est induit par les actions de l'utilisateur. En effet lorsque l'utilisateur ne fait aucune action dans son environnement, c'est qu'il lui convient. C'est donc un signe que le système remplit son rôle. Au contraire, si l'utilisateur continue à effectuer des actions dans l'environnement (pour le modifier) alors, le système est en échec, car il ne satisfait pas l'utilisateur.

6.2.2 Fonctionnement du système

Malgré que le système proposé pour la détection des préférences utilisateurs semble adéquat et qui soit possible de le développer durant le stage, mes encadrants de l'IRIT ont proposé d'utiliser un autre système déjà développé qui nécessiterais moins de code, et laisserai plus de temps au développement.

Le système que nous avons utilisé est similaire à un séparateur linéaire.

Le système étant basé sur celui présenté par [3], des reliquats du vocabulaire persistent. A l'origine il était utilisé pour détecter des situations anormales chez les personnes âgées pour détecter si ces personnes étaient en danger ou non. Cependant, dans notre cas, nous n'allons pas l'utiliser pour détecter quand effectuer une action. Ainsi, pour une action données, une situation anormale est une situation nécessitant d'effectuer une action tandis qu'une situation normale est une situation ne nécessitant pas d'action.

Il faut ici faire la différence entre le système global, qui est responsable de répondre à la problématique, et les sous systèmes, qui décomposent la tâche. A partir d'ici nous ne considérons plus le système global mais plutôt une instance d'un sous système. L'agrégation de ces sous-système va permettre la résolution du problème.

Chacun des sous-systèmes, va être responsable de détecter les situations dans lesquelles il faut effectuer une action. Le système global devra être capable de détecter l'ensemble des actions qui lui sont disponibles et instancier un sous-système pour chacune de ces actions. Pour des raisons de temps, le stage fut concentré sur la mise en place d'un seul sous-système.

Données en entrée

Le système fonctionne avec des contextes, le but étant de séparer les contextes en deux ensembles, il utilise en entrée les valeurs de tous les capteurs disponibles ainsi que les valeurs des actuateurs. Ces valeurs constituent un vecteur avec :

- C_i : Valeur du capteur i
- V_j : Valeur de l'actuateur j
- CG_k : Valeur du capteur global k

Le vecteur obtenu est selon la forme suivante : $(C_1, ..., C_i, A_1, ..., A_j, CG_1, ..., CG_k)$

Données en sortie

Comme énoncé précédemment, le système n'est pas réellement conscient de l'action qu'il prédit. En effet, il fonde son raisonnement sur le feedback qu'il reçoit et l'action qu'il prédit est transparente pour lui.

Sa manière de communiquer est donc binaire. A chaque situation rencontrée et portée à sa connaissance, le système va émettre une alerte ou indiquer que la situation est normale. Cette information devra ensuite être interprétée, et reliée à l'action que prédit ce système, pour comprendre la décision.

Méthode de résolution

Pour résoudre le problème de séparation linéaire, le système modélise la fonction linéaire de séparation avec des agents.

Le but du système est de séparer des contextes. Un contexte est un vecteur qui présente une valeur pour chacune des perceptions du système. A chaque nouveau contexte rencontré, le système émet un avis sur ce nouveau contexte et lance une alerte ou non.

Afin de choisir entre un contexte lançant une alerte et un contexte normal, le système définit une somme pondérée des valeurs du contexte. Si cette somme est supérieure à un seuil fixé empiriquement, alors une alerte est lancée, sinon le système annonce que la situation est normale.

Pour calculer cette somme pondérée des poids sont conservés à chaque résolution. Ces poids sont les outils qui permettent au système de séparer les situations anormales des autres. Chaque poids est attribué à une des perceptions, et pondérera toujours la valeur de cette perception à chaque évaluation.

L'évolution de ces poids va permettre au système de déplacer la somme pondérée de chaque situation et ainsi influencer sur sa prédiction, et mieux répondre au besoin.

Avantages et limites

Cette technique de résolution présente différents avantages. Tout d'abord, l'ajout ou le retrait de nouvelles perceptions n'empêche pas le système de continuer à fonctionner et ne nécessite pas une réinitialisation des connaissances. C'est une caractéristique importante puisque le contexte d'implémentation du système est un système IoT. C'est un environnement dans lequel la panne d'un capteur, la volatilité des capteurs connectés et leur mobilité peut créer une certaine instabilité de l'information, et il faut pouvoir maintenir un niveau de service à chaque instant.

Ensuite, le système fonctionnant sur la base d'un séparateur linéaire, certes multi-agent, conserve beaucoup de leurs avantages. Notamment la rapidité de convergence. Un système linéaire converge rapidement lorsque des exemples critiques et problématiques sont résolus. Nous nous attendons donc à ce que ce système converge rapidement. Cela permet une mise en place rapide, et une entrée en fonction sans délai, dès lors que les informations sont disponibles.

Cependant, le système étant linéaire, et comme nous avons pu le constater au travers de nos tests, il est incapable, dans l'état, d'apprendre des comportements complexes (c'est à dire non linéaires). Cela réduit grandement son utilité dans le cadre des environnements socio-techniques, car les humains ont souvent des comportements complexes à décrire même pour des tâches simples.

Enfin, il n'échappe pas au travers des systèmes multi-agent, l'émergence. Même si l'émergence est un outil fantastique, qui permet la création de comportement complexe via des interactions simples et compréhensibles par les développeurs, elle peut parfois poser des problèmes. Notamment en termes de coût de débogage. Comme le comportement émerge, il n'y a pas de ligne de code particulière qui engendre le bug, et il faut donc déployer des efforts de log et d'analyse pour situer le problème.

Feedback

À chaque nouveau cas rencontré, le système reçoit un feedback. Chaque situation rencontrée doit présenter un feedback. Les feedbacks sont enregistrés afin de pouvoir utiliser leur connaissance dans les résolutions futures. Ainsi, à chaque instant le système fait évoluer ses poids de manière à ce que la connaissance qu'il a de son environnement soit toujours respectée. Plus de détails sur ce comportement par la suite.

On distingue quatre types de feedback :

- *VraiPositif* : Lorsque le système prédit que la situation actuelle *est* une situation qui doit soulever une alerte, et que le feedback *confirme* cette décision.

- *VraiNegatif* : Lorsque le système prédit que la situation actuelle *n'est pas* une situation qui doit soulever une alerte, et que le feedback *confirme* cette décision
- *FauxPositif* : Lorsque le système prédit que la situation actuelle *est* une situation qui doit soulever une alerte, et que le feedback *ne confirme pas* cette décision.
- *FauxNegatif* : Lorsque le système prédit que la situation actuelle *n'est pas* une situation qui doit soulever une alerte, et que le feedback *ne confirme pas* cette décision.

6.2.3 Agents

Ce système est donc un système multi-agent qui reproduit les fonctionnalités d'un séparateur linéaire tout en lui apportant certains avantages forts des systèmes multi-agent. On diffère deux types d'agents.

Agent Poids

Cet agent est le responsable du poids lié à chaque perception. Voici les étapes de son cycle de décision :

- Perception : à chaque cycle l'agent reçoit des messages provenant des agents contraintes. Ces messages contiennent une demande de modification du poids, soit une augmentation soit une réduction.
- Décision : face à tous ces messages, l'agent se retrouve fréquemment en situation contradictoire. Certains messages demandent une augmentation, d'autres une réduction. Il décide quel message traiter, en sélectionnant celui provenant d'un agent contrainte avec la criticité la plus élevée.
- Action : l'agent fait progresser le poids dans le sens choisi, à l'aide d'un mécanisme adaptatif. Ce mécanisme est chargé de faire progresser le poids plus vite dans le cas de même actions répétées, et moins vite dans le cas de suite d'action contraires.

Agent Contrainte

Définition

Cet agent est le représentant d'une situation rencontrée. A chaque instant le système ne conserve que les agents contraintes les plus porteurs d'information (les plus critiques).

Chaque agent contrainte retient les valeurs de l'environnement lors de sa création ainsi que la qualité de cette situation (alerte ou pas). A chaque instant il est donc capable, à l'aide des valeurs de l'environnement sauvegardées et les valeurs des agents poids, de calculer quelle serait la réaction du système si cette situation se représentait. Ainsi, il a pour but de toujours faire correspondre cette réaction théorique du système avec la qualité de la situation qu'il représente.

Types d'agents Contrainte

On distingue deux types d'agents contraintes dans le système. Chaque agent contrainte représente en fait une inégalité stipulant : selon les valeurs de la situation que je représente, la somme pondérée que je calcule doit se trouver de tel côté du seuil". Ainsi, en fonction du côté du seuil désiré par l'agent, il définit son type : *Supérieur* ou *Inférieur*.

On peut faire correspondre chacun des types de feedback présenté précédemment avec un type d'agent contrainte.

- *Supérieur* : Un agent souhaitant être au dessus du seuil est un agent représentant une situation avérée comme anormale, ainsi les feedback *VraiPositif* et *FauxNegatif* entraîneront la création d'agent contrainte de ce type.
- *Inférieur* : Un agent souhaitant être en dessous du seuil est un agent représentant une situation avérée comme normale, ainsi les feedback *FauxPositif* et *VraiNegatif* entraîneront la création d'agent contrainte de ce type.

Mesure de criticité

Pour rester du bon côté du seuil, il dispose d'une mesure afin de faire valoir son état au système et aux autres agents, la criticité. Cette valeur est la distance de sa somme pondérée (ses valeurs sauvegardées et les poids actuels portés par les agents poids) avec le seuil du système (déterminé à l'initialisation du système). Lorsque la criticité est négative c'est que l'agent est positionné du bon côté du seuil, si elle est positive, il est mal placé. Les agents *Supérieurs* auront donc une criticité positive si en dessous du seuil, et inversement, les agents de type *Inférieur* auront donc une criticité positive si au dessus du seuil.

Cycle de vie

A chaque feedback reçu, un agent contrainte est créé. Il n'est pas forcément conservé, mais une fois créé, le système le compare aux autres agents présents. Si le nouvel agent a un niveau de criticité supérieur à un autre agent déjà présent ce dernier est supprimé, et le nouveau est conservé. Le système ne conserve à tout instant qu'un certain nombre d'agents contraintes afin d'éviter l'explosion combinatoire. Le but de ce mécanisme est de conserver les informations importantes que le système a acquies avec le temps tout en permettant d'en apprendre de nouvelles. Il est admis que si un agent est proche du seuil, après équilibrage, il est probable que des modifications des poids vont fortement l'affecter et risquent de le déplacer de l'autre côté. Comme cet agent est proche, il représente un cas très porteur d'informations, et symbolise bien la limite entre les deux ensembles.

Afin de faire fléchir les poids des agents poids en sa faveur il dispose d'une action d'envoi de message. Voici son cycle de décision :

- Perception : l'agent récupère les poids actuels des agents poids, et calcule sa somme pondérée. Puis à l'aide de cette somme pondérée, il détermine sa valeur de criticité.
- Décision : si cet agent est le plus critique de son type, il va donc préparer un message contenant sa valeur de criticité, ainsi que la direction dans laquelle il veut évoluer. Ce message sera envoyé à chaque agent poids. Il joint aussi, une valeur spéciale pour chacun des poids : l'influence. Cette valeur représente la proportion du terme lié à une perception, dans la somme pondérée. Certaines perceptions n'étant pas importantes parfois pour la détection d'alerte, leur influence sera faible.
- Action : l'agent émet son message vers les agents poids.

6.2.4 Mécanisme d'apprentissage

C'est lors des interactions entre les Agents Contraintes et les Agents Poids que le système apprend. Nous allons ici détailler ce mécanisme plus en détail avec des exemples.

A chaque instant, toutes les situations représentées par les agents Contraintes sont "projetées" sur un espace 2D (figure 6.2), via somme pondérée. Les situations au-dessus d'un seuil sont considérées comme anormales, et le système générera une alerte si elles se représentent, tandis que celles en dessous sont normales et ne généreront pas d'alerte.

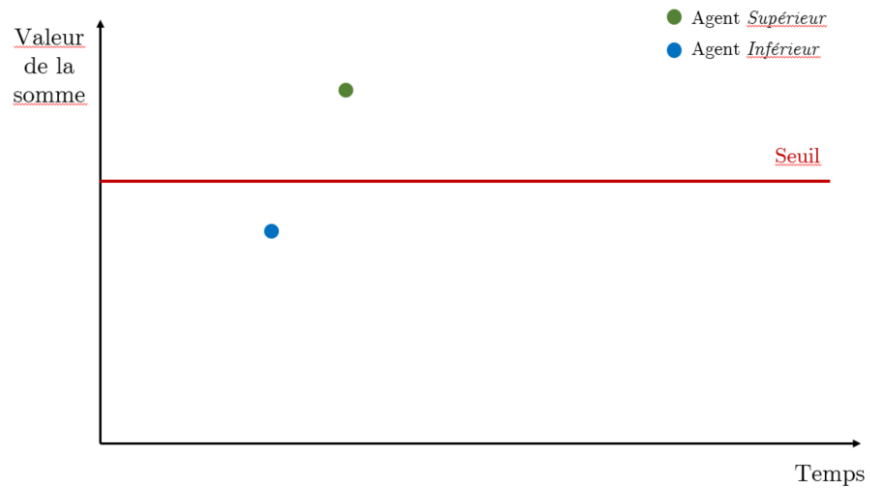


FIGURE 6.2 – Projection des contextes

Lorsqu'une nouvelle situation se présente elle est projetée sur dans l'espace et le système réagit à sa position par rapport au seuil. Dans le cas présenté par la figure 6.3, le système va lever une alerte.

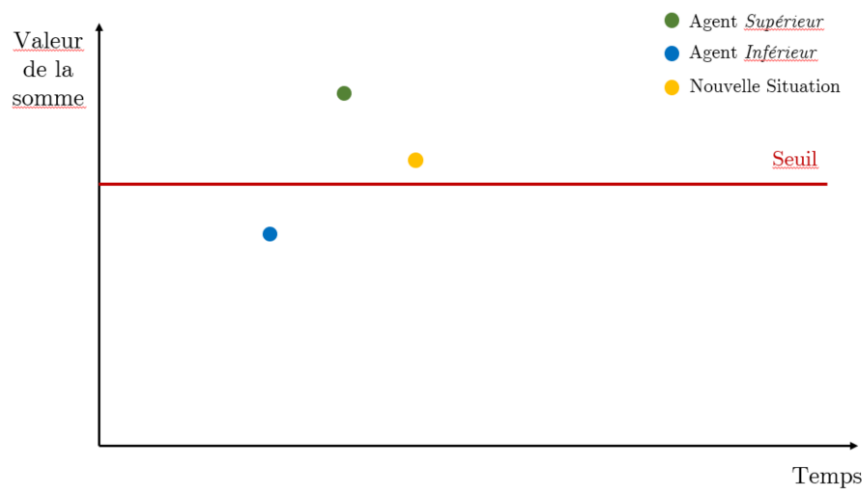


FIGURE 6.3 – Projection d'un nouveau contexte

Cependant, considérons que le système se soit trompé, et que cette situation ne nécessite pas d'action, alors elle sera catégorisée par un agent *Inférieur*. Ce nouvel agent, va faire valoir sa criticité et les agents poids vont modifier leurs valeurs pour lui permettre de se retrouver en dessous du seuil. A noter que les modifications des poids entraînent un déplacement de tous les Agents Contrainte dans l'espace (figure 6.4)

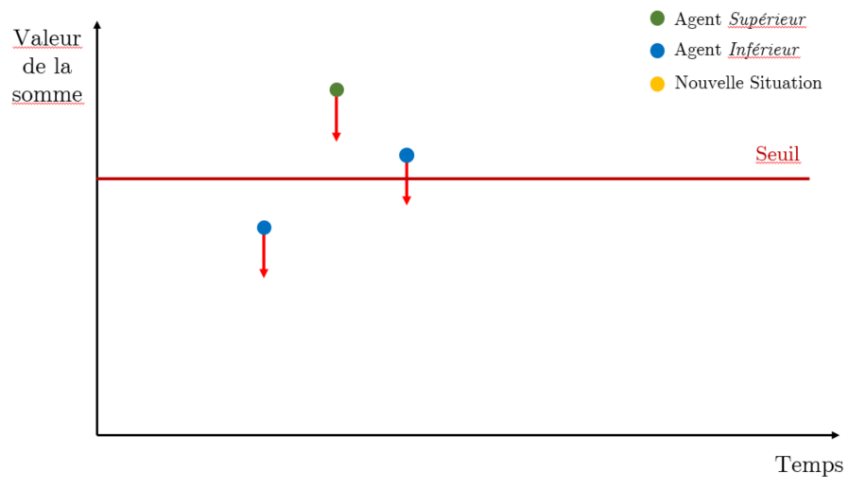


FIGURE 6.4 – Déplacement des contextes par modification des poids

Bien entendu, comme les valeurs des situations ne sont pas les mêmes, parfois la modification d'un poids ne déplacera presque pas un Agent Contrainte tandis que d'autres seront grandement affectés.

6.2.5 Adéquation fonctionnelle

Il semble évident qu'une fonction linéaire n'est pas suffisante pour séparer toutes les actions. Il est donc nécessaire d'avoir un système par action, et chaque système sera chargé d'apprendre à détecter lorsqu'il faut effectuer son action.

Cependant nous avons réalisé, vers la fin du stage, que cette manière de procéder n'était pas suffisante, et qu'un tel système ne serait pas suffisant pour résoudre notre problème.

Prenons un exemple de situation dans laquelle le système est incapable de séparer les situations en deux ensemble. Supposons que le système fonctionne avec deux perceptions, la température et le niveau de CO2. Le but du système est de détecter les situations dans lesquelles il faut ouvrir la fenêtre (un autre système similaire serait nécessaire afin de savoir quand fermer la fenêtre).

Comme le système tente de reproduire le comportement d'un utilisateur, qu'on suppose régi par un système de règles, il est possible, dans notre environnement de test de générer le feedback utilisateur. Voici les règles choisies :

- Règle 1 : Température >23
- Règle 2 : Température >15 && CO2 >1500

Si on représente ces deux ensembles dans la figure 6.5 on peut remarquer qu'une fonction linéaire ne peut pas séparer ces deux ensembles. Un seul système n'est donc pas suffisant pour capter et apprendre le comportement de l'utilisateur sur une action.

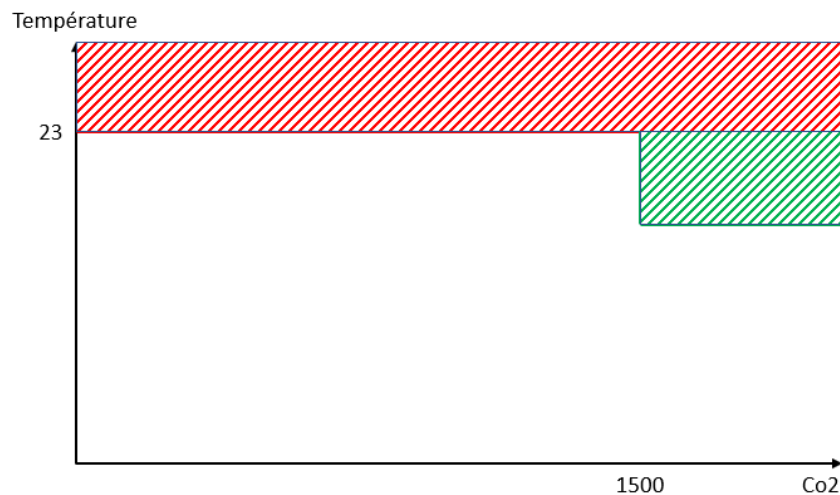


FIGURE 6.5 – Représentation de l'ensemble des états nécessitant d'ouvrir la fenêtre

6.2.6 Coopération entre systèmes

La solution retenue pour faire face à ce problème est d'utiliser plusieurs systèmes. L'intuition est basée sur le fait que généralement, des fonctions non linéaires (comme celle qui sépare les deux ensembles de contextes) sont approchables par une combinaison de fonctions linéaires. C'est ce comportement que nous avons reproduit.

Le système développé par [3] ne permet pas de telles interactions entre les systèmes. Il nous a fallu concevoir cette coopération et la développer sur les bases du système existant.

Un système représentant une fonction linéaire, il nous faut plusieurs itération de ce système pour approcher un comportement complexe. Les systèmes multi-agents sont souvent candidat à l'implémentation de tels mécanismes. Lorsque le système rencontre un état de blocage qu'il ne peut résoudre seul, il fait appel à d'autres afin de le résoudre. Ici, il faut que le système se multiplie afin que suffisamment de fonctions linéaires soient présente, et proposent un approche du comportement suffisante. Pour des raisons de temps nous n'avons pas pu implémenter ce mécanisme. Ainsi le système démarre avec un nombre de système défini heuristiquement. La possibilité d'implémenter le démultiplication des système dans le futur n'est pas exclue, et le développement actuel prendre en compte cette ouverture.

Nous sommes donc dans le cas où plusieurs systèmes doivent coopérer afin d'approcher une fonction non linéaire afin des fonctions linéaires. Tous les systèmes fonctionnent de la même manière. Ils sont tous soumis aux même événements et ont les même perceptions de l'environnement.

Lorsqu'un système rencontre un cas qui le déséquilibre et qui n'est pas solvable (c'est-à-dire, qu'après équilibrage des poids, il n'est pas possible de satisfaire tous les agents contraintes), il doit coopérer avec les autres afin de pouvoir prendre en compte la situation courante, sans pour autant perdre des connaissances déjà stockées dans le système.

Afin de prendre en compte la connaissance de la situation actuelle, il faut conserver l'agent contrainte créé. Cependant, conserver cet agent contrainte déséquilibre le système et le rend insolvable. Pour savoir quel système doit le conserver, on choisit celui avec la mesure de criticité la plus faible. Dans le cas ou plusieurs systèmes possèdent une mesure de criticité égale, un système est choisi au hasard.

Le système ainsi choisi conserve le nouvel agent contrainte, tandis que les autres le suppriment. Ce système cependant est insolvable car il vient de prendre en compte une situation contradictoire, car non séparable linéairement des autres. Il va donc supprimer tous les contextes qui empêchent cet agent contrainte d'être bien placé.

Lorsque ces agents contraintes ont été supprimés, le système peut lors retrouver son équilibre. La connaissance portée par ces agents contraintes n'est cependant pas perdue car, toujours présente dans les autres systèmes.

Finalement, il n'est pas envisageable de laisser un système se vider de l'intégralité de ses agents d'un seul type, afin de satisfaire une seule situation. Aussi, le nombre d'agents supprimables est limité à 20% du nombre total d'agents contraintes conservés.

Le fonctionnement décrit dans cette partie n'est pas développé entièrement encore. Le stage terminant le 28 septembre, cela constitue le reste du travail.

6.2.7 Initialisation du système

A l'initialisation du système, il n'existe aucun agent contrainte. A la rencontre de la toute première situation (nécessitant une action ou non), le système va s'adapter à la topologie des données qu'il rencontre. Il va créer un agent poids par valeur reçue, et va les initialiser aléatoirement.

Cependant il va aussi créer un agent contrainte pour cette situation. Cet agent étant le seul présent dans le système, il va être le seul à influencer le système, et va totalement le faire progresser dans son sens. Suite à une longue période avec un seul type d'agent, les poids seront totalement conditionnés par ces agents là, et si une situation de l'autre type apparaît, le système va devoir déployer un gros effort pour converger à nouveau.

Le système tel que nous l'avons récupéré ne propose pas de solution à ce problème et nous avons conçu et développé un mécanisme multi-agent afin de pallier à cette difficulté d'initialisation.

Pour faciliter et surtout accélérer cette première convergence, nous avons créé des agents contraintes virtuels. Ces agents virtuels ne vont pas vraiment représenter des situations réelles, mais vont renvoyer une criticité fixe, de manière à ce que les agents non virtuels ne s'éloignent pas trop du seuil pendant l'initialisation du système.

Au début, le système possède donc deux agents virtuels, un de chaque type. Lorsque qu'une situation crée un agent contrainte du même type qu'un des agents virtuels, celui-ci est alors supprimé.

6.3 Sélection du système développé

Nous avons choisi de développer la seconde conception étudiée durant le stage. Plusieurs raisons ont amené ce choix malgré que les deux conceptions semblent adéquates fonctionnellement pour notre problème.

Tout d'abord, cette conception étant basée sur le travail d'un membre de l'IRIT et de l'équipe SMAC, il nous est possible de récupérer des morceaux de codes. Ainsi, nous avons pu grandement réduire la durée de développement et directement travailler à l'implémentation du système dans notre cas.

Aussi, cette approche semblait moins complexe à mettre en place, et réduit les chances de débordements sur le planning. Le but étant de pouvoir tester le système dans un environnement réel avant la fin du stage.

6.4 Outil de visualisation

Dans l'état où nous l'avons récupéré, le système ne disposait pas d'outils de visualisation ni d'évaluation. J'ai développé plusieurs solutions afin de pouvoir mieux évaluer le système lors de nos tests.

Certaines interactions étant complexes, il n'est pas possible de les visualiser autrement que textuellement, ainsi un système de log pour chaque type d'interaction s'est révélé utile pour analyser les modification du système ainsi que les interaction internes. Malgré qu'un fichier log ne soit pas très lisible et compréhensible, il apporte un support à l'analyse lorsque le système rencontre des bugs et des erreurs.

Ensuite, un système modulaire d'affichage de graphe a été mis en place. Ce système permet à l'exécution de déterminer les valeurs que l'on souhaite afficher durant la résolution du système, et des graphiques mis à jour en temps réel sont affichés. Cela permet de mieux comprendre l'évolution dans le temps du système.

Chapitre 7

Évaluation

7.1 Génération des données

Afin de tester le système, nous disposons d'une salle équipée de capteurs et d'actuateurs réels. Le but du stage était de pouvoir tester le système dans un environnement réel. Cependant, il existe une couche technique importante afin de pouvoir se connecter à ces salles ce qui représente plusieurs semaines de développement. Aussi, la période de vacances implique qu'aucun étudiant n'est présent dans ces salles, ce qui empêche l'expérimentation.

Nous avons donc principalement utilisé des données générées à l'aide de l'outil TSimulus. Cet outil est un générateur de données réalistes aléatoires et temporelles. Il permet de générer des données basées sur des modèles réels, tout en appliquant de l'aléatoire pour simuler les variations réelles de ces valeurs.

Par exemple pour simuler un capteur de température, nous partons d'une base mensuelle qui est extraite de moyennes météorologiques réelles.

- Janvier : 5.0 C°
- Février : 6.7 C°
- Mars : 8.9 C°
- Avril : 11.1 C°
- Mai : 15.0 C°
- Juin : 18.9 C°
- Juillet : 21.1 C°
- Aout : 21.1 C°
- Septembre : 18.3 C°
- Octobre : 13.9 C°
- Novembre : 8.9 C°
- Décembre : 6.1 C°

Puis nous appliquons une variation journalière avec une granularité de deux heure.

- "00 :00 :00" : -3 C°
- "02 :00 :00" : -3.9 C°
- "04 :00 :00" : -5 C°
- "06 :00 :00" : -4.6 C°
- "08 :00 :00" : -5.7 C°
- "10 :00 :00" : -2.2 C°
- "12 :00 :00" : 1 C°
- "14 :00 :00" : 3 C°
- "16 :00 :00" : 2.3 C°
- "18 :00 :00" : 0.9 C°
- "20 :00 :00" : -2.3 C°
- "22 :00 :00" : -2.7 C°

Il est ensuite possible d'appliquer une fonction de bruit par dessus ces données générées. Ce qui nous permet d'obtenir une série réaliste.

Pour nos différents essais nous avons travaillé sur des séries de deux ans, avec une granularité de deux heures.

Finalement, nous générons aussi un feedback synthétique. Pour ce faire nous définissons un système de règles sensé reproduire le système de règles d'un humain. Pour des buts expérimentaux, nous avons rendu ce système de règles plus simple. Le feedback est ensuite utilisé par le système pour évaluer ses actions.

7.2 Critères d'évaluation

La plupart des mesures d'analyse statistiques ne sont pas applicables aux systèmes multi-agents, et n'apportent que très peu dans notre démarche expérimentale. Souvent les systèmes multi-agents sont évalués expérimentalement, car il est très difficile de mesurer et d'évaluer la qualité de l'émergence.

Ainsi, aucune mesure particulière n'était observée afin d'évaluer le système. Cependant différentes métriques sont très pratiques afin de détecter les erreurs et l'incapacité du système à apprendre le comportement que l'on cherche à cerner.

Tout d'abord le nombre d'erreurs par rapport au feedback est très révélateur. En comparant les nombre de fois où le système reçoit des feedback *FauxPositif* ou *FauxNegatif* aux nombres de fois où le système reçoit des *VraiPositif* ou *VraiNegatif* on peut quantifier le nombre d'erreurs. Le système étant capable d'apprendre en un nombre très faible d'exemples, nous attendons un nombre d'erreur très faible.

7.3 Tests avec un capteur

Les premiers tests ont été effectués avec un seul capteur en entrée. Ces tests ont pour but de s'assurer du bon fonctionnement du code récupéré.

Pour ce premier test, nous avons décidé d'utiliser un capteur de température généré. Le but de ce test est de s'assurer que les agents soient correctement générés, et que les poids des agents soient modifiés correctement.

Le but du système est d'apprendre quand ouvrir la fenêtre. Comme présenté, les données générées contiennent un feedback synthétique dont la règle est la suivante : $Temperature > 23$

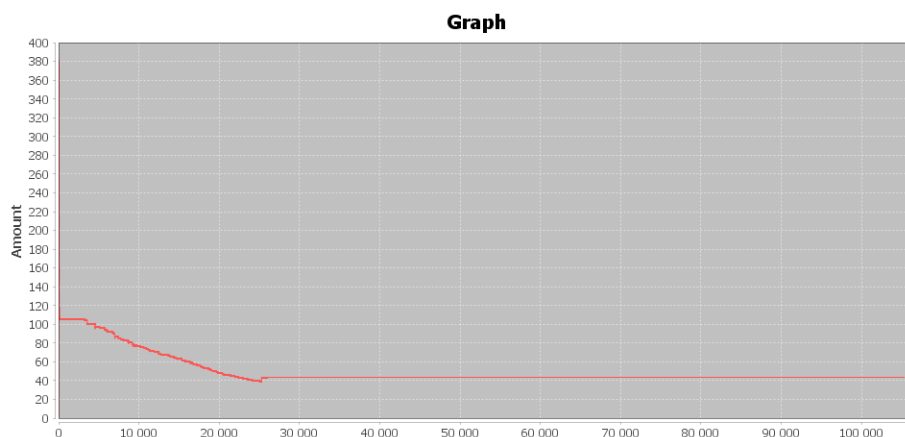


FIGURE 7.1 – Valeur de l'agent poids température

On peut constater sur la figure 7.1 que le poids de la température évolue rapidement au début. En effet, les températures commençant au mois de Janvier, nous ne rencontrons pas de cas dans lesquels il faille ouvrir la fenêtre. Cependant, tant que le système n'a pas rencontré de cas dans lesquels ouvrir la fenêtre, il ne fera pas l'erreur de l'ouvrir.

```
TRUEPOSITIVE 2824
TRUENEGATIVE 102422
FALSEPOSITIVE 15
FALSENEGATIVE 2
```

FIGURE 7.2 – Feedbacks reçus pendant le test avec un capteur

Le soudain arrêt de modification du poids s'explique par la rencontre des premiers cas dans lesquels il faut ouvrir la fenêtre. Si l'on regarde la figure 7.2, on constate que le système ne fait que très peu d'erreurs. Ces erreurs sont au début de son apprentissage, et lors de la rencontre des premiers cas dans lesquels il fallait ouvrir la fenêtre.

En conclusion, le système semble réagir comme souhaité dans ce cas là et propose un niveau de service à la hauteur de nos attentes.

7.4 Tests avec plusieurs capteurs

Les tests avec plusieurs capteurs sont les plus proches des situations réelles que nous visons avec le système. En effet ici la décision du système doit être basée sur l'observation de plusieurs critères.

Les capteurs utilisés sont un capteur de température, un capteur de Co2 et un capteur de présence.

Le but du système est toujours d'apprendre lorsqu'il faut effectuer l'action "ouvrir la fenêtre" ou non. Le système de règles utilisé pour générer le feedback est le suivant :

- R1 : $Temperature > 23$
- R2 : $Temperature > 15 \wedge Co2 > 1500$
- R3 : $Co2 > 2000$

La formule pour la décision finale est :

$$D = R1 || R2 || R3$$

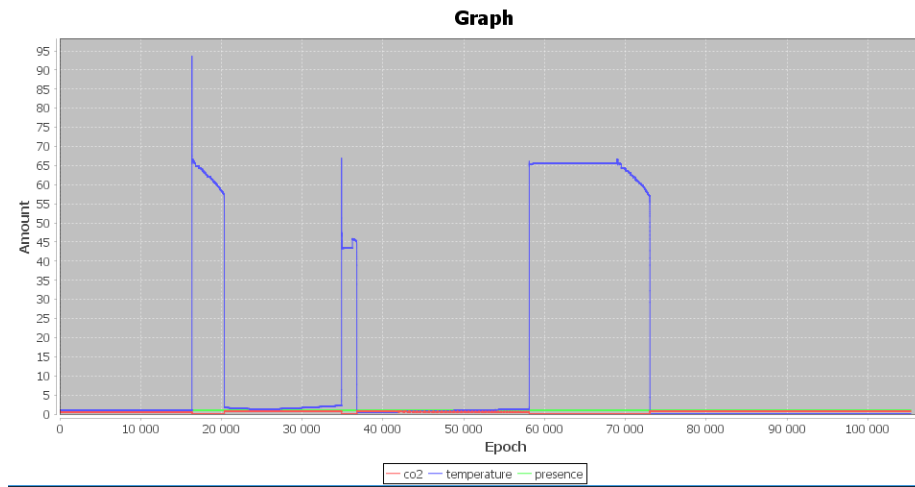


FIGURE 7.3 – Valeurs des agents poids

Lorsque l'on analyse les valeurs des poids dans ce cas là (figure ??, on constate que le système a fait face à plusieurs conflits. Le comportement étant trop complexe pour une fonction linéaire, il n'a en fait appris qu'une partie de ce comportement à chaque fois. Ainsi, lorsqu'il rencontre une autre expression de ce comportement, il est forcé "d'oublier" ce qu'il savait afin de satisfaire le fragment qu'il observe.

```
TRUEPOSITIVE 4780
TRUENEGATIVE 83473
FALSEPOSITIVE 15870
FALSENEGATIVE 1140
```

FIGURE 7.4 – Feedbacks reçus pendant le test avec plusieurs capteurs

Ceci se reflète dans le nombre d'erreurs faites par le système durant le test comme le montre la figure 7.4. Dans ce cas là, le système ne fournit pas un niveau de service acceptable. C'est pourquoi nous avons entamé le développement de modifications, comme la coopération entre plusieurs systèmes.

Chapitre 8

Conclusions

8.1 Conclusion

Durant ce stage, j'ai conçu un l'architecture d'un système multi-agent d'intelligence ambiante. Le but de ce système est de maximiser le confort de l'utilisateur en apprenant ses préférences à travers ses actions dans son environnement. Ce système est composé de cinq modules principaux, le premier est celui chargé de virtualiser le monde réel. Le second est celui chargé d'apprendre les préférences des utilisateurs et de choisir l'action à réaliser pour les faire respecter. Le module d'apprentissage qualitatif est chargé de comprendre les intentions des utilisateurs à travers leurs actions. Enfin le module d'apprentissage Quantitatif est chargé de prédire l'état du système après une action.

Nous aurons étudié deux implémentations possibles du module Préférences Utilisateurs et choisi celle qui nous paraissait la plus pertinente pour démarrer le projet. Des modifications sur ce systèmes restent encore possible afin d'augmenter sa pertinence et son efficacité.

Afin de l'améliorer il est possible de lui fournir des entrées plus pertinente avec un mécanisme de pré-traitement des données, ou de le coupler avec d'autres systèmes d'informations du même système IoT. Ce système représente un base de départ solide pour des travaux similaires.

Dans le mois restant, l'implémentation de la coopération entre système sera terminée d'être implémentée et pourra être testée.

8.1.1 Bilan

Pour l'entreprise, ce projet n'est que le début d'une volonté plus grande. En effet les systèmes IoT manquent d'applications et ce projet est le début d'un travail autour des applications de l'IoT. Il constitue une première prise en main des systèmes multi-agent pour Sogeti, et c'est un paradigme d'intelligence artificielle qu'ils souhaitent poursuivre d'explorer. Finalement, le travail fourni propose une base solide afin de continuer à explorer l'implémentation de systèmes multi-agents dans les systèmes IoT et pourra être amélioré ou utilisé comme référence.

Pour le laboratoire, ce stage représente le début d'une collaboration avec l'entreprise, notamment à travers le projet neOCampus. Cela aura permis également de tester un de leur systèmes dans nouveau cas d'utilisation, et les résultats de ces essais seront bénéfiques à la recherche sur des projets similaires.

8.1.2 Bilan personnel

Ce stage fut la suite logique d'une partie de mes enseignements sur les systèmes multi-agents. J'ai pu mettre en œuvre les méthodes d'analyse de problèmes sous le paradigme de ces systèmes, mais aussi de m'adonner à leur conception dans un contexte plus concret. J'ai pu également comparer ce paradigme aux techniques d'apprentissage plus traditionnelles comme ceux vu en cours, tels que les réseaux de neurones.

Par rapport au stage que j'ai précédemment réalisé, celui-ci m'a certainement accordé plus de liberté dans mes choix et décisions. Je n'ai pas seulement suivi des consignes de développement mais j'ai pu concevoir le système, proposer mes solutions et décider desquelles développer. La durée de six mois a aussi permis d'avancer plus que dans mes précédentes expériences.

Dans le cadre de mon projet professionnel, ce stage m'a conforté dans mon choix d'effectuer une thèse industrielle. En effet la rencontre entre les contraintes de l'industrie et les possibilités de la recherche et des techniques de l'état de l'art, représente un véritable challenge que je trouve très stimulant. La possibilité de poursuivre ce travail dans le cadre d'une thèse CIFRE m'a été proposé, mais j'ai également le projet de partir faire cette thèse à l'étranger.

Bibliographie

- [1] Jacques Ferber. Multi-agent system : An introduction to distributed artificial intelligence. 4, 01 2001.
- [2] Marie-Pierre Gleizes, Valérie Camps, Jean-Pierre Georgé, and Davy Capera. Engineering systems which generate emergent functionalities. In Danny Weyns, Sven A. Brueckner, and Yves Demazeau, editors, *Engineering Environment-Mediated Multi-Agent Systems*, pages 58–75, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [3] Nicolas Verstaevl Carole Bernon Jean-Pierre George Marie-Pierre Gleizes. A self-organized learning model for anomalies detection application to elderly people. 2001.
- [4] Valérian Guivarch. *Prise en compte de la dynamique du contexte pour les systèmes ambiants par systèmes multi-agents adaptatifs*. PhD thesis, Université Paul Sabatier, Toulouse, 2014.
- [5] Sebastien Mazac. *Approche décentralisée de l'apprentissage constructiviste et modélisation multi-agent du problème d'amorçage de l'apprentissage sensorimoteur en environnement continu : application à l'intelligence ambiante*. PhD thesis, Université Claude Bernard - Lyon 1, 2015.
- [6] Julien Nigon. *Apprentissage artificiel adapté aux systèmes complexes par auto-organisation coopérative de systèmes multi-agents*. PhD thesis, Université Paul Sabatier, Toulouse, 2017.
- [7] S. J. Russel and P. Norvig. *Artificial Intelligence : A Modern Approach*. Pearson Edition, 2003.
- [8] Nicolas Verstaevl. *Self-organisation of robotic devices through demonstrations*. PhD thesis, Université Paul Sabatier, Toulouse, 2016.

Appendices

Date de début	Date de fin	Phase	Numéro de sprint	Sous-numéro de sprint	Sous phase	User story
03/04/18	13/04/18	vision	0	0	vision	Etude documentaire multi-agents
16/04/18	27/04/18	vision	0	1	vision	Etude documentaire multi-agents
30/04/18	11/05/18	functional design	1	0	functional design	Conception système
14/05/18	25/05/18	functional design	2	0	build	Mise en place générateur de données
				1	functional design	Conception système
28/05/18	08/06/18	technical design	3	0	validation	Tests du générateur de données
				1	functional design	Prise en main du code multi-agent
				2	build	Tests du code multi-agent
11/05/18	22/06/18	technical design	4	0	build	Test du code multi-agent
				1	technical design	Prise en main du code multi-agent
25/06/18	06/07/18	technical design	5	0	build	Refactoring code multi-agent
				1	technical design	Ajout de fonctionnalité, agents initiaux statiques
09/07/18	20/07/18	technical design	6	0	validation	Test du système avec plus de données en entrée
				1	functional design	Etude parallélisation avec données générateur
				2	technical design	Etude implémentation parallélisation avec données générateur
23/07/18	03/08/18	technical design	7	0	technical design	Etude parallélisation
				1	functional design	Correction bugs système multi-agent
				0	functional design	Conception systèmes coopératifs
06/08/18	17/08/18	build	8	1	vision	Rédaction rapport
				0	vision	Rédaction rapport
20/08/18	31/08/18	validation	9	0	vision	Développement systèmes coopératifs
03/09/18	14/09/2018	deploy	10	0	build	Test systèmes coopératifs
09/03/18	14/09/2018	deploy	10	0	deploy	Test en situation réelle du système